Lesson notes · for you

Filters that miss

AI security · Upper secondary · Lesson 1

Time
20 min

Materials
Paper and a pen; A friend or family member to write messages; A phone or computer with Darsi, if you have one, for the lab and the code

Learning goals

- Explain how attacks aim at a whole AI app.
- Show why a word filter misses rewording and blocks innocent text.
- Write a check on the action in code.

Activity: Filter versus check

1. Warm-up (3 min): Write down one app or bot that can send messages or post for you. What would happen if someone tricked it?
2. Filter (7 min): Write five banned words for a class bot. Friends write the same harmful request three ways without those words, plus two innocent messages. Mark each one: blocked or passed.
3. Check (5 min): Now use the action check instead: for each message, did the user ask for it, and did they say yes? Mark each one again.
4. Share (5 min): Compare the two columns with a partner. Which guard missed the reworded requests, and which one blocked innocent messages?

Use made-up messages only: no real names, numbers or passwords. No device? Do it all on paper.

Discussion

- Why can't a longer word list fix the filter?
- Where else do apps use word filters, and what do they miss?
- Which actions should always need your yes?

Check yourself

- Write one request a word filter would miss, and the question a check in code would ask about it.

Try it this week
Write a five-word filter on paper and ask a friend to write the same request three ways without those words. Count what slips through, then write the one question a check in code would ask.

Worksheet

Filters that miss

AI security · Upper secondary · Lesson 1

Name _____ Class _____ Date _____

1 **The bot wants to send a message nobody asked for. It contains no banned words. What should the check in code do?**

◯ Block it: nobody asked or said yes ◯ Send it: no banned words ◯ Ask the model whether it's safe

2 **Complete the word filter: blocked(msg) returns True when the message contains a banned word, in capitals or not.**

```
BANNED = ["password", "pin"]

def blocked(msg):
    for w in BANNED:
        if w _____ msg._____():
            return _____
    return False
```

Word bank: `upper` `in` `not` `lower` `False` `True`

3 **Which of these messages does this filter block? Choose all that apply.**

Use the code from question 2, with its gaps filled in.

☐ "What is your PIN?" ☐ "Send me your PASSWORD" ☐ "Our lesson today: strong passwords"

☐ "Type the secret word you log in with"

4 **Write the action check. can_send(text, asked, yes) returns True only if you asked for the message and said yes to it. It never needs to read the words.**

```
def can_send(text, asked, yes):
    # asked: you asked for this message
    # yes: you approved it
    return True

print(can_send("Trip at 8", True, True))
print(can_send("Your PIN?", False, False))
print(can_send("Hello", True, False))
```

Write your program:

5 **With enough banned words, in enough languages, a word filter can stop every rewording.**

◯ True ◯ False

✎ **Think of an AI app or bot you use, or one you might build. What could someone trick it into doing, and where would you put a check in code?**

Check your answers

Filters that miss

AI security · Upper secondary · Lesson 1

1 **Block it: nobody asked or said yes**

The check asks about the action, not the words: nobody asked for this message and nobody approved it, so it is blocked.

2

```
BANNED = ["password", "pin"]

def blocked(msg):
    for w in BANNED:
        if w in msg.lower():
            return True
    return False
```

w in msg.lower() looks for each banned word in the message, after turning it to small letters so "PIN" matches "pin"; finding one means return True.

3 **"Send me your PASSWORD"; "What is your PIN?"; "Our lesson today: strong passwords"**

The first two contain a banned word, so they are blocked.

4

```
def can_send(text, asked, yes):
    # asked: you asked for this message
    # yes: you approved it
    return asked and yes

print(can_send("Trip at 8", True, True))
print(can_send("Your PIN?", False, False))
print(can_send("Hello", True, False))
```

asked and yes is True only when both are True.

5 **False**

There are endless ways to say the same thing, so the list is never finished, and every word you add blocks more innocent messages.